

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 1 of 14

SDK API User Manual for Micro Family

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 2 of 14

1. Introduction

a. Purpose:

The purpose of this document is to brief developers regarding the usage of the SDK API, which is used to communicate with the Micro Family systems.

b. Scope:

The scope of this manual is to cover the basic usage of the SDK API, and is limited to several use cases examples, and will not include all possible function calls and different use cases the SDK API supports.

This Manual will also include an explanation regarding the SDK API Demo application.

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 3 of 14

2. SDK API Overview

a. Description:

SDK is an API designed to communicate and control over any Micro-family system.

The SDK handles only the control channel and does not support video streaming.

b. OS Support

The API supports both Windows and Linux Operating Systems.

c. API File Structure:

The API is written in C language and composed of 2 files:

- i. SDKAPI.c – source code and implementation of different function calls.
- ii. SDKAPI.h – include file for the developer to include in the target application, with each file description.

d. The SDK API does not handle by itself any thread and may be implemented in user thread or a big while loop, according to the hosting software architecture.

e. Communication brief:

- i. The Micro-family systems are controlled using UDP communication, with different messages running between the system and host (SDK API).
- ii. Function types:
 1. Host to System: all Host-To-System messages are identified as "Set" command, and the function calls will always start with "Set" prefix. For Host-To-System message The API parses internally the requested operation from the functions and send the relevant command to the system.
 2. System to Host: all System-To-Host messages are identified as "Get" reports, and the function calls will always start with "Get" prefix. For System-To-Host message The API parses the data sent from the system to the host, placing the received data in relevant structure for the different functions to retrieve the data from the structs field.

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 4 of 14

3. SDK API Architecture

a. Architecture overview:

- i. The API is comprised of different objects, each object is an instantiation of a struct.
- ii. Each object can hold data either to send to the system, or to get from the system.
- iii. Each object has different data field representing different parameters of the system.
- iv. Each "Set" or "Get" function accesses a specific field of an object.
- v. Send or receive mechanisms affecting an object or multiple objects.

b. "Set" functions:

- i. All "Set" functions are used to send data from the host to the remote system.
- ii. Set functions name breakdown:

1. int Set<Struct><Field>(<Argument>)

- a. int: return 0 if function executed successfully.
Notice that function returns 0 if the command was successfully sent, and not if the system executed it.
To check for execution or change in parameters a relevant "Get" command must be called.
- b. "Set" is used to identify that the function will send data to system.
- c. <Struct>: The group that this command is linked to.
- d. <Field>: The parameter name to modified.
- e. <Argument>: The value to set.
- f. For example: int SetRateAngularVelocities_X(float AngularVelocities_X)
Will be used to set an angular velocity for the X axis of the system.

iii. "Set" functions mechanism:

Whenever a Set function is called, it will update an internal object's field. If the defined "SEND_ON_SET", the function call will also send the entire object to the system, and the returned value is addressed to the status of the UDP transaction.

If the "SEND_ON_SET" is not defined, it will return 0, and only the object field will be updated, to send the data the "SDKUpdateRemote ()" function must be called.

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 5 of 14

c. "Get" functions:

- i. All "Get" functions are used to retrieve the latest data from the API's Objects different fiel.
 - The "Get" functions do not handle any UDP transaction by themselves, to get the data from the system a "SDKSyncLocal ()" command must be called.
- ii. "Get" function name breakdown:
 1. <Return type> Get<Struct><Field>()
 - a. <Return type>: type of returned value.
 - b. "Get" is used to identify that the function will return data of a parameter.
 - c. <Struct>: The group that this command is linked to.
 - d. <Field>: The parameter name to modified.
 - e. For example: float GetTIRReportMsgFOV() will be used to get the current FOV of the thermal camera.
- iii. "Get" functions mechanism:
Whenever a "Get" function is called, it will retrieve a value from the API's internal relevant object's field and return it.

d. General functions:

- i. int SDKInitializeCommunication(struct MessagesTransmitEnableStruct data):
used to initialize a UDP connection to the remote system.
 1. This function is used to connect to the remote system and must be called prior to any other API function.
 2. Takes as an argument a predefined " struct MessagesTransmitEnableStruct" argument, this argument is used to initialize the object " MessagesTransmitEnable" for any "SDKUpdateRemote()" call.
 3. Will return 0 for success.
- ii. int SDKUpdateRemote()
 1. This function is used to send data from the host to the system.
 2. The objects sent are as defined by the value of "MessagesTransmitEnable"
 3. This function is used whenever required to send several objects at once.
 4. Will return 0 for success.
- iii. int SDKSyncLocal(uint16_t timeout)
 1. This function is used to receive data sent from the remote and store it in the relevant objects.
 2. Must be called to update local data.
 3. Takes an argument of uint16_t timeout, determining the milli secon time the API will wait for new data to arrive.
Recommended timeout is around 10 milli secon.
 4. Will return 0 for success.

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 6 of 14

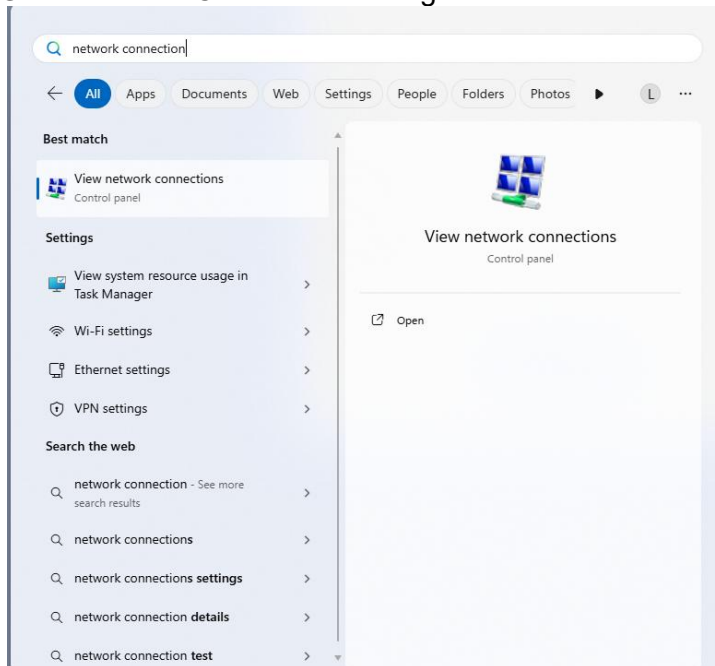
e. API parameters:

The API have several parameters, manifested in several defines which can be found at the SDKAPI.h file:

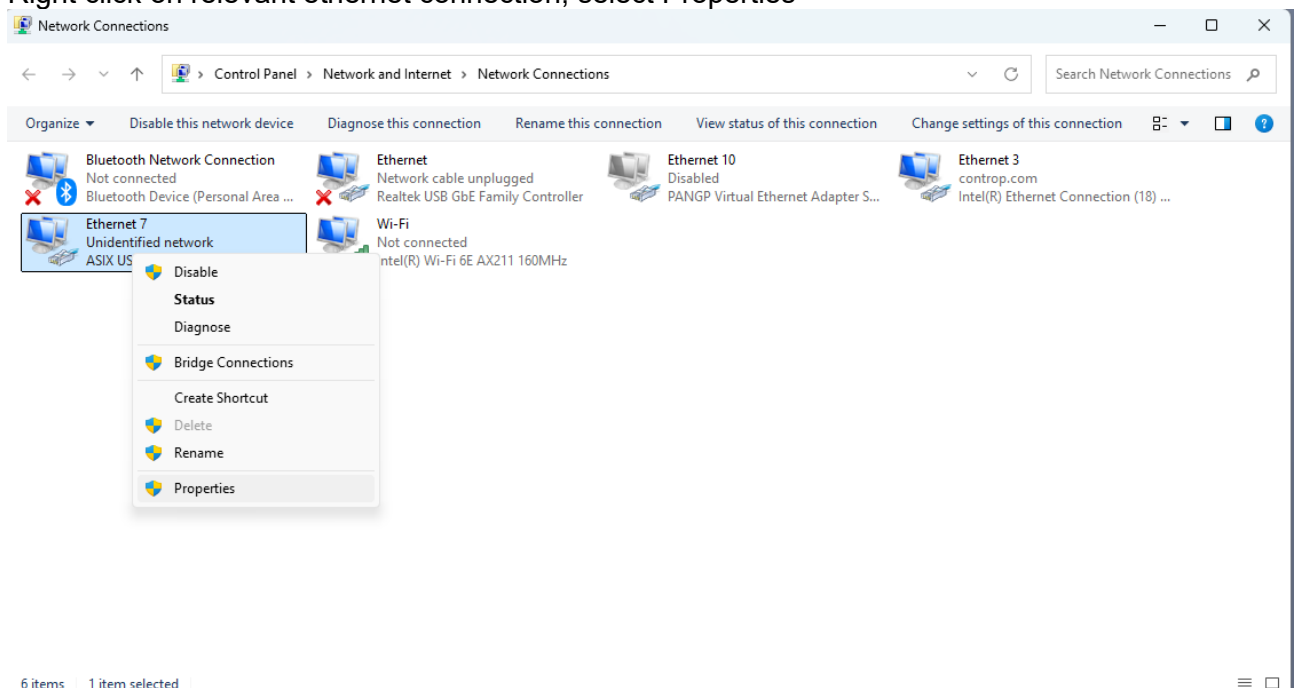
- #define HOST_IP "1.1.1.2" : Used to set the Host IP address.
- #define REMOTE_IP "1.1.1.3 " : Used to set the Device IP.
- #define COMMUNICATION_PORT 1024 : Used to set data uplink port.
- #define LOCAL_PORT 8080 : Used to set data downlink port.
- #define SEND_ON_SET:
 - If defined, each "Set" function will transmit a command to the device.
 - If not defined, each "Set" function call will only modify data internally within objects instantiated internally within the API, but will not be transmitted to the device.

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 7 of 14

- For setting IP:
 - Go to Network Connection settings via Start Menu



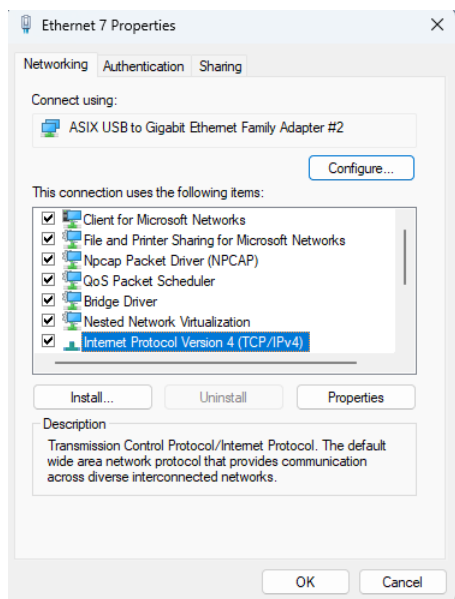
- Right click on relevant ethernet connection, select Properties



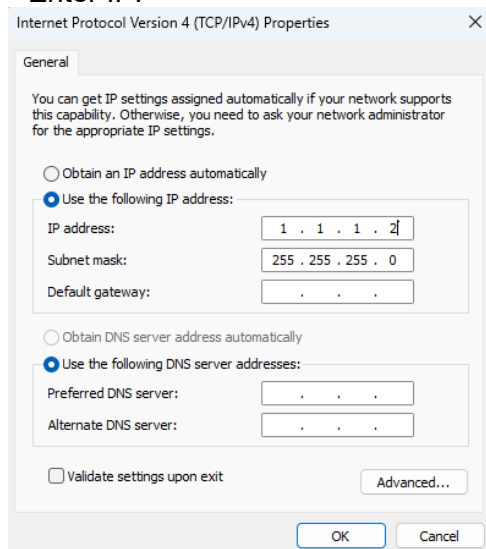
- Select Internet Protocol Version 4 (TCP/IPv4)

~ 7 ~

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 8 of 14



- Enter IP:

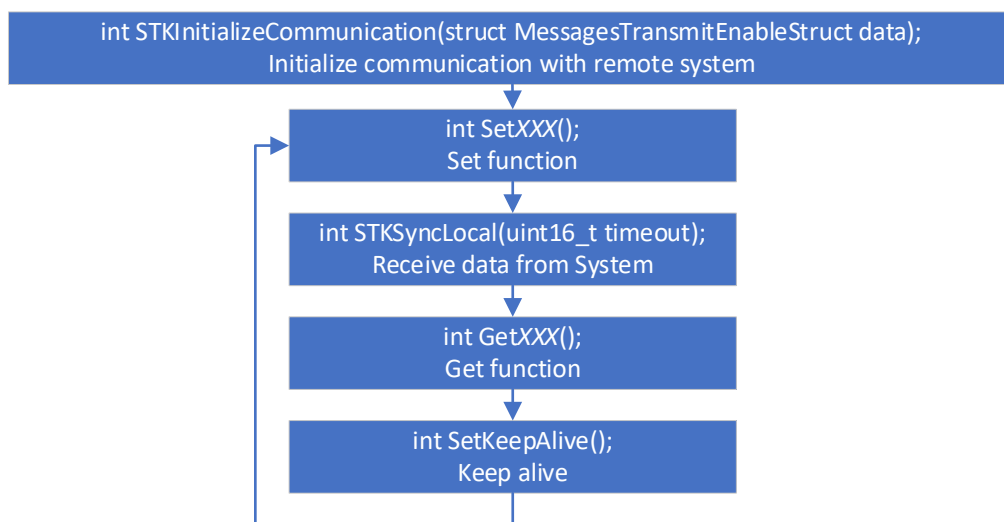


- Press OK and then OK again

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 9 of 14

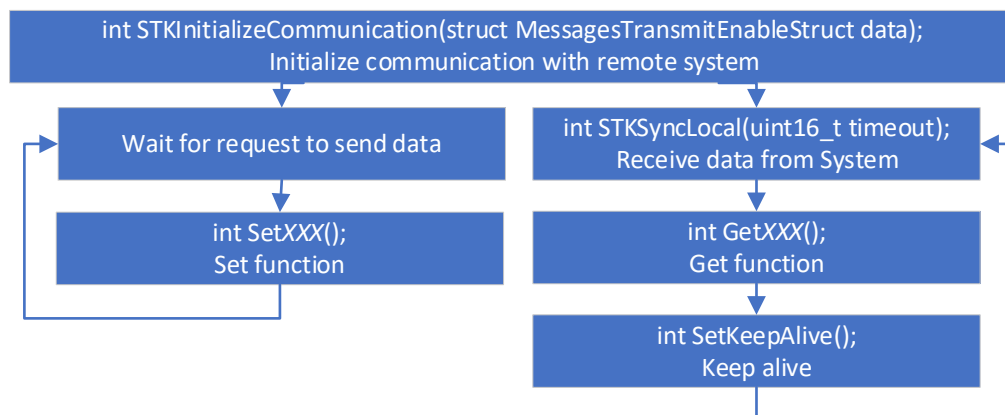
4. SDK API implementation recommendation

- i. This is a general example of a communication flow and function calls using the SDK API in a big while loop architecture.
 1. Initialize communication to the system using "int SDKInitializeCommunication(struct MessagesTransmitEnableStruct data);"
 - This step is mandatory, and cannot be skipped, any function call done prior to this function success will result in failure.
 2. Send parameters to the system using "int SetXXX();", for example, set zoom, change to thermal, move orientation etc.
 3. Update local data objects using "int SDKSyncLocal(uint16_t timeout);"
 - This step is mandatory for updating the local data, any "Get" function call will return data based on the available local data.
 4. Get a parameter using "int GetXXX();". For example, get current FOV, system orientation, etc.
 5. Send a keep Alive signal using "int SetKeepAlive();".
 - This step is mandatory, if a Keep Alive signal is not sent once every 500 milli secon the system will assume communication lost and will switch to its default parameters.
 6. Repeat Set function, sync local data, get local data, set Keep Alive signal.



	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 10 of 14

- b. This is a general example of communication flow and function calls using the SDK API in a 2 threa architecture.
- i. Prior to any function call, a " SDKInitializeCommunication(struct MessagesTransmitEnableStruct data);" function call must be successfully executed.
 - ii. after communication initialization done, we can spawn 2 threa
 1. Set Thread – in charge of sending data from the host to the remote system upon request.
 - a. Wait for a request to send data to host.
 - b. Call an "int SetXXX();" , for example, set zoom, change to thermal, move orientation etc.
 - c. Cyclic wait for request, send data etc.
 2. Sync / Get / Keep Alive thread – in charge to cyclic update objects, local variables and send a keep alive signal to the remote system.
 - a. Update local data objects using "int SDKSyncLocal(uint16_t timeout);" every 10 milli secon.
 - b. Get a parameter using "int GetXXX();" . For example, get current FOV, system orientation, etc.
 - c. Send a keep Alive signal using "int SetKeepAlive();" every 500 milli secon.
 - d. Cyclic update local data, gets relevant parameters, send keep alive, etc.



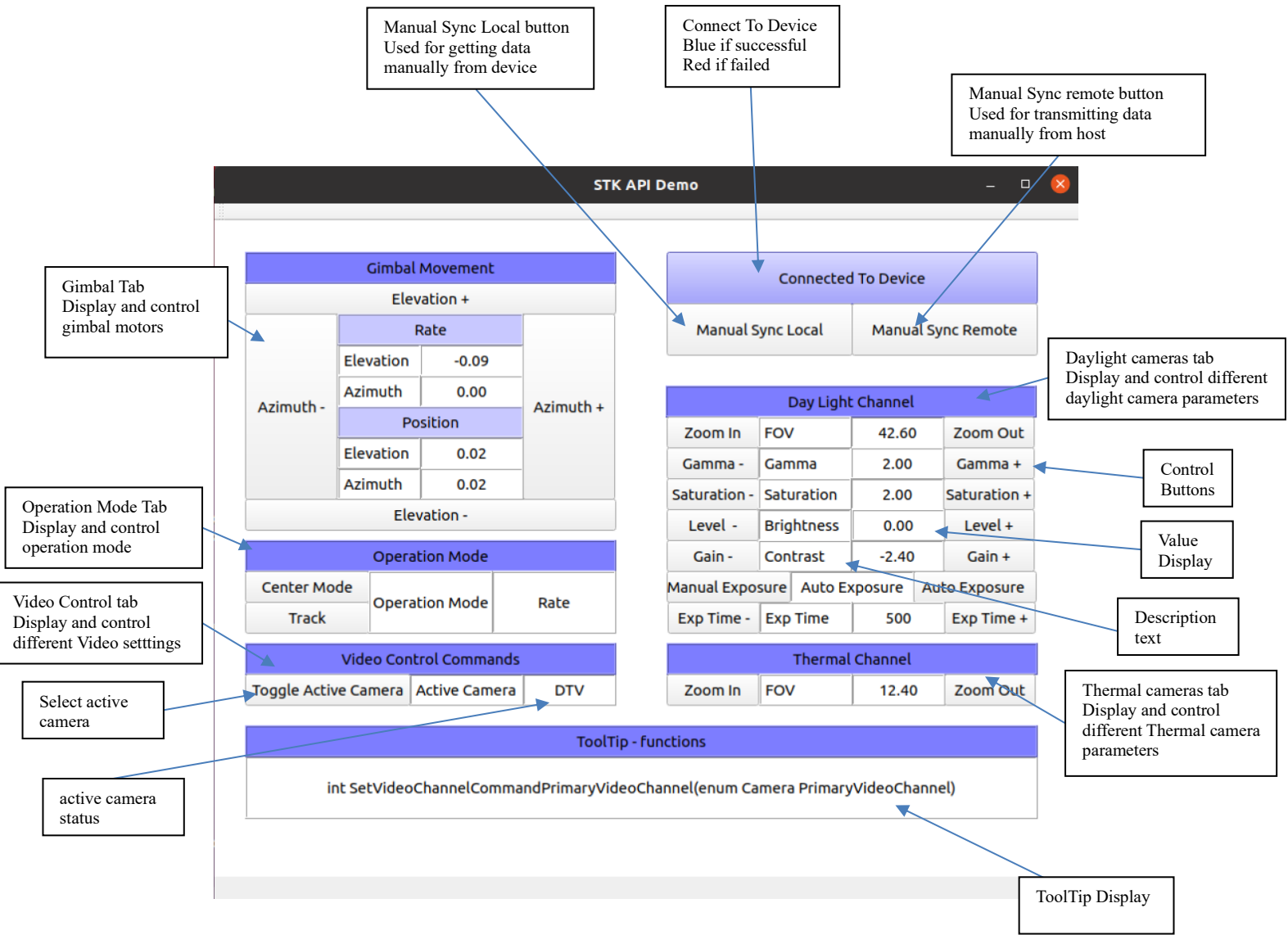
	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 11 of 14

5. SDK API Demo

- a. The SDK Demo is a reference design, with basic implementation of functions and QT based GUI, designed to give the developer a starting point in understanding the way of implementing the SDK API.
- b. The SDK Demo is using a main thread, used to read buttons and control the different "Set" function calls, while a secondary thread is used to cyclic call "SDKSyncLocal()" function and call various "Get" functions, also this thread is used to cyclic call the function " SetKeepAlive()" every 500 milli secon.
- c. The Demo is designed to work in Linux OS, tested in Ubuntu 20.04 with QT version 4.11.0
- d. The Demo composed of several files:
 - i. main.cpp – instantiating the GUI thread and the data sync thread.
 - ii. mainwindow.ui – GUI define file.
 - iii. mainwindow.cpp - defines the connections and relationship between the SDK API and the GUI (buttons, text boxes, etc..).
 - iv. worker.h – defines the data recieve thread, each 10ms assert SDKSyncLocal function, extract relevant data and display it.
Also, this thread asserts SetKeepAlive() function.
 - v. HelperFunctions.cpp – helper functions source file.
 - vi. HelperFunctions.h – helper functions include file.
 - vii. SDKAPI.h – includes file for SDK API.
 - viii. SDKAPI.c – source file for SDK API.
- e. Demo usage:
 - i. Open the Demo executable.
 - ii. Press on "Connect To Device" button, if connected successfully the button will turn blue, if failed the button will turn red.
 - iii. The values in the text boxes should be updated and not equal to 0.
 - iv. Day Light Channel is implemented in full for the demo, while the Thermal channel only implements zoom control.
 - v. Control is available for orientation of the system in rate mode, while the text boxes indicate both rate and position values.
 - vi. To modify video stream between DTV (Day Light) and TI (Thermal) please use the Video Control Comman tab.
 - vii. When hovering with the mouse over a text box or a button, the function used will be displayed on the screen.

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 12 of 14

SDK Demo application for Linux



	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 13 of 14

6. SDK API Demo for Windows.

- a. The SDK Demo is a reference design, with basic implementation of functions on C++ Visual Studio GUI, designed to give the developer a good starting point in understanding the way of implementing the SDK API.
- b. The Demo is designed to work in Windows, tested in Win11 with VS2022
- c. The Demo composed of several files:
 - i. MyForm.cpp – is the main GUI form.
defines the connections and relationship between the SDK API and the GUI (buttons, text boxes, etc..).
Also, there is a rx thread, each 33ms assert SDKSyncLocal function, extract relevant data and display it
 - ii. SDK.h – include file for SDK API.
 - iii. SDK.c – source file for SDK API.
- d. Demo usage:
 - i. Open the Demo executable.
 - ii. Press on "Connect To Device" button, if connected successfully the button will turn blue, if failed the button will turn red.
 - iii. The values in the text boxes should be updated and not equal to 0.
 - iv. Day Light Channel is implemented in full for the demo, while the Thermal channel only implements zoom control.
 - v. To modify video stream between DTV (DayLight) and TI (Thermal) please use Video Control Comman tab.
 - vi. When hovering with the mouse over a text box or a button, the function used will be displayed on the TootTip text box. You can copy the written function for searching at c/h file.

	Unclassified	June 11, 2025
Document: SDK_User_Manual		
Document No.:	Rev: 1.0	Page: 14 of 14

SDK Demo application for windows

